

9. ОРГАНИЗАЦИЯ РАЗРАБОТКИ

9.1. Описание выбора методологии управления процессом разработки

Командная разработка подразумевает взаимодействие нескольких человек при реализации программного продукта. Чтобы организовать управление процессом разработки были придуманы различные методы. Применение данных методов зависит от входного состояния проекта и подготовленности команды к процессу разработки.

В целом среди методологий можно выделить следующих представителей:

1. Каскадная модель (она же “водопадная” – waterfall).
2. Итерационные модели.
3. Инкрементная модель.
4. Спиральная модель.

По большому счету все методологии можно разделить на две больших группы: последовательные и итерационные.

9.2. Waterfall (каскадная модель)

Основная суть модели **Waterfall** (рис. 9.1) в том, что этапы зависят друг от друга и следующий начинается, когда закончен предыдущий, образуя таким образом поступательное (каскадное) движение вперед.

Параллелизм этапов в каскадной модели, хоть и ограничен, но возможен для абсолютно независимых между собой работ. При этом интеграция параллельных кусков все равно происходит на каком-то следующем этапе, а не в рамках одного.

Команды разных этапов между собой не коммуницируют, каждая команда отвечает четко за свой этап.

Недостатками этой модели являются получение результата по прохождению всех этапов и сложность выявления ошибок. Возвращаться назад трудно. Не понятно, что возвращать: если произошел сбой на каком-то этапе, его последствия видны только в конце.

Для заказчиков данная модель выглядит линейно и со стороны достаточно просто: из завершеного этапа проектирования следует программирование, а затем тестирование – и так шаг за шагом пока не будет достигнута финальная точка и цель, ради которой ведется разработка.

Данная модель понятно и чисто укладывается в документы, например в договора и роадмапы при наличии четко обозначенных контрольных точек. В любой момент времени можно легко понять была ли пройдена та или иная точка контроля или нет, и соблюдены ли сроки. По этим причинам долговременные и особо крупные проекты, рассчитанные на десятилетия и вовлечение большого

числа организаций-участников, руководствуются преимущественно waterfall.

Однако представление о простоте каскадной модели является иллюзорным. Оно появляется из-за ограниченного видения клиентом всего процесса, ведь данная модель не подразумевает вовлечение заказчика в детали процессов разработки, и демонстрирует понятный и конечный результат работы только на контрольных точках и в конце проекта.

В реальности каскадную модель нельзя назвать простой, на практике ею сложно управлять. Внесение заказчиком значительных изменений в процессе разработки по waterfall или срабатывание серьезных не предусмотренных проектом рисков несут разрушительный характер для всего процесса - модель приходится перестраивать, графики перепланировать.

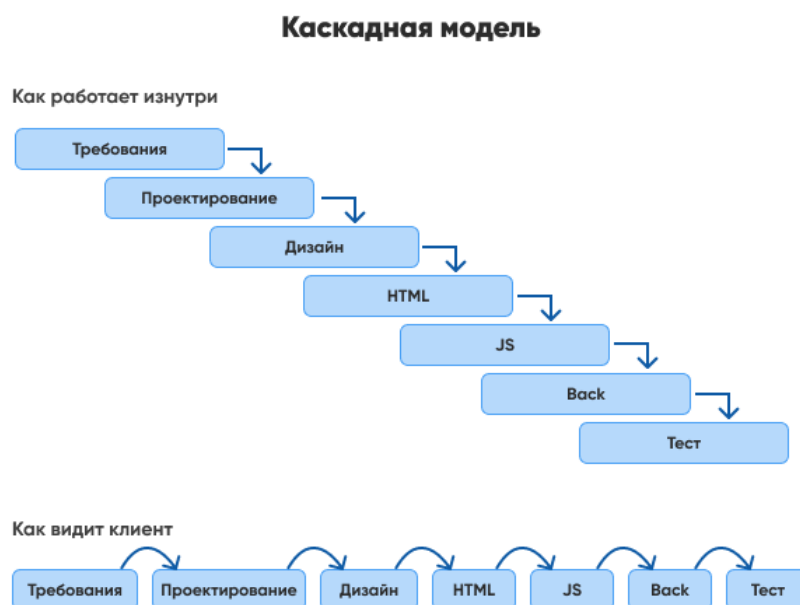


Рисунок 9.1. Каскадная модель

9.3. Итерационная, спиральная и инкрементная модели

Итерационная модель (рис. 9.2) предполагает разбиение проекта на части (этапы, итерации) и прохождение этапов жизненного цикла на каждом из них. Каждый этап является законченным сам по себе, совокупность этапов формирует конечный результат.

Поясним разбиение на этапы на бытовом примере. Допустим, нам нужен стол в гостиную.

- на первом этапе мы сделаем столешницу, ножки и скрепим их так, чтобы стол стоял;
- на втором, укрепим и покрасим его;

- а на третьем, накроем скатертью и купим подходящие к нему стулья.

На каждой итерации мы работали с одним и тем же продуктом и в конце каждой итерации получали результат, которым можно пользоваться (естественно, с определенными ограничениями).

С каждым этапом разработка приближается к конечному желаемому результату или уточняются требования к результату по ходу разработки, и соответственно в любой момент текущая итерация может оказаться последней или очередной на пути к завершению.

Данный подход позволяет бороться с неопределенностью, снимая ее этап за этапом, и проверять правильность технического, маркетингового или любого другого решения на ранних стадиях.

Использование итерационной модели снижает риски глобального провала и растраты всего бюджета, получение несинхронизированных ожиданий и ошибочного понимания процессов как клиентом, так и каждым участником команды разработки. Оно также дает возможность завершения разработки в конце любой итерации (в каскадной модели вы должны прежде завершить все этапы).

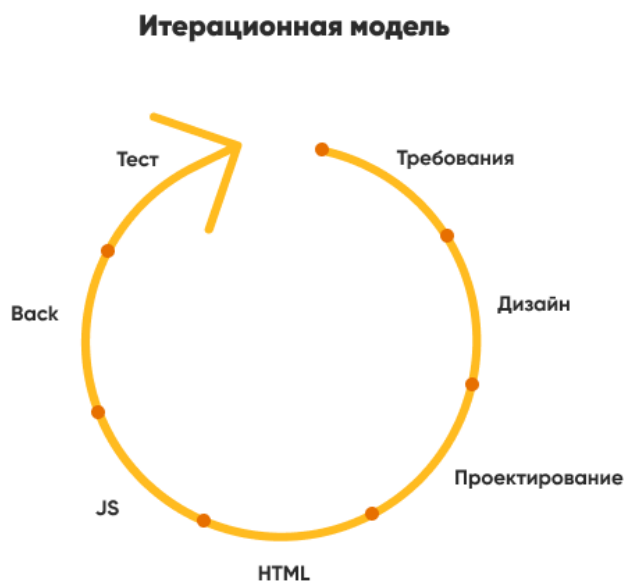


Рисунок 9.2. Итерационная модель

Спиральная и инкрементная модели являются видами итерационной модели жизненного цикла.

9.3.1. Спиральная модель

Все этапы жизненного цикла при спиральной модели (рис. 9.3) идут витками, на каждом из которых происходят проектирование, кодирование, дизайн,

тестирование и т. д. Такой процесс отражает суть названия: поднимаясь, проходит один виток (цикл) спирали для достижения конечного результата. Причем не обязательно, что один и тот же набор процессов будет повторяться от витка к витку. Но результаты каждого из витков ведут к главной цели.

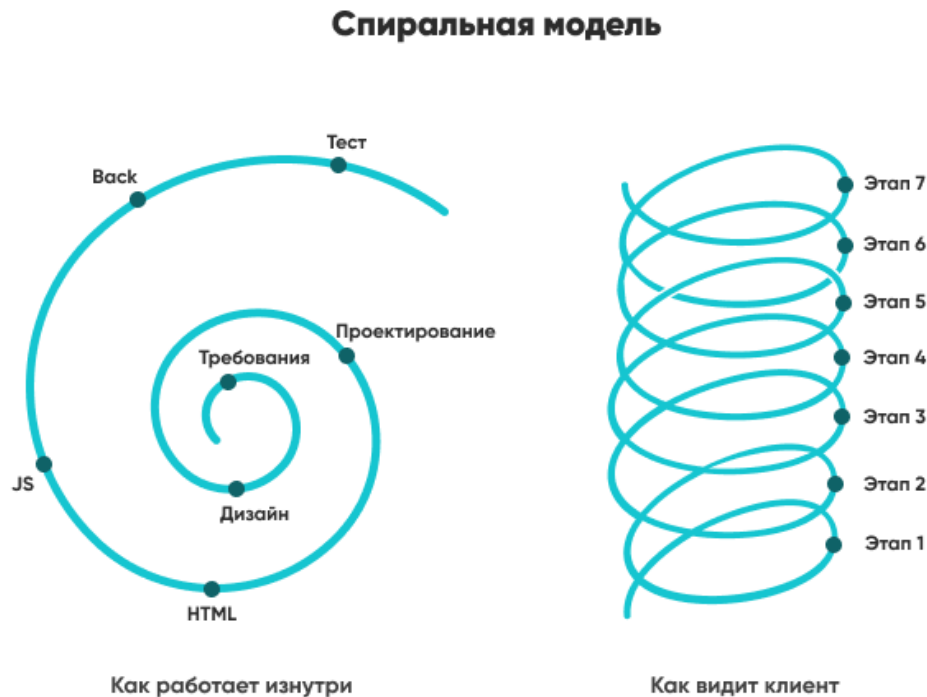


Рисунок 9.3. Спиральная модель

9.3.2. Инкрементная модели

Принцип, который лежит в основе инкрементной модели, подразумевает расширение возможностей, достраивание модулей и функций приложения. Буквальный перевод слова инкремент: «увеличение на один». Это «увеличение на один» применяется в том числе для обозначения версий продукта.

Если в каскадной модели, по сути, есть два состояния продукта: «ничего» и «готовый продукт», то с появлением итерационных моделей стало применяться версионирование продукта. Каждая итерация обозначается цифрой: 1,2,3 и соответственно продукт после каждой итерации имеет версию с соответствующим номером: v.1, v.2, v.3. Числами после слова версия обозначают масштабные изменения в ядро продукта.

А когда одна из версий эксплуатируется, следующая, учитывая недочеты предыдущей, только планируется или уже разрабатывается, а улучшения заказчику и пользователю хочется доставить прямо сейчас, тогда появляются минорные версии. Туда попадают изменения, которые не влияют на ядро разработки и

представлены как под-версии 1.1,1.2,1.3 или релизы 1.1.1, 1.1.2 и т.п.

В совокупности такие поэтапные релизы приводят к полноценной версии 2.0.

9.4. Agile и Lean: принципы разработки ПО

9.4.1. Agile

Agile — набор принципов гибкой разработки (всего их 12) и идей. Основные идеи Agile:

- люди и взаимодействие важнее процессов и инструментов;
- работающий продукт важнее исчерпывающей документации;
- сотрудничество с заказчиком важнее согласования условий контракта;
- готовность к изменениям важнее следования первоначальному плану.

Один из принципов - взаимодействие - подразумевает, что заказчик взаимодействует с командой, команда с заказчиком - все между собой. Это позволяет обмениваться опытом между участниками команды и клиентом и участвовать каждому из них в принятие решений. За счет такого подхода снижаются риски потери времени и денег и повышается способность команды решать сложные нестандартные задачи с высокой степенью неопределенности.

Однако взаимодействия всех и со всеми может вылиться в хаос, что влияет на все сферы разработки. Поэтому используя Agile нужно понимать ограничения: команды должны быть небольшие, участники должны быть компетентные и мотивированные, итерации короткие с максимально понятными целями, установлены четкие ограничения по времени и конечный результат должен быть очевидным.

Agile прекрасно управляется с неопределенностью, предопределяя будущее на более короткий период. Правило такое: чем выше неопределенность - тем короче итерация, причем ее длительность может быть даже в рамках 24 часов, как и происходит на хакатонах. Вначале каждой итерации неизбежно выполняется контроль, ретроспектива, оценка и анализ результатов, планирование следующей итерации.

Во внутреннем планировании и в продуктовой разработке без этого принципа и элементов Agile не обойтись.

9.4.2. Lean

Идея подхода **Lean** в том, что мы бережливо относимся к ресурсам (в том

числе времени) и решаем задачи самым простым способом. Например: мы не делаем весь продукт, чтобы понять, что он никому не нужен – мы делаем лендинг и форму подписки и даем на него рекламу, чтобы проверить что этот продукт вызывает интерес клиентов и принять осознанное решение о необходимости разработки.

Когда доходит до разработки продукта, или делается какое-то улучшение, производственное или инженерное, мы сначала делаем его MVP (minimum viable product). Термин MVP сейчас широко распространён и применяется повсеместно, но он родился именно из Lean подхода. MVP это такая версия продукта, которая выполняет свою главную функцию и при этом её не отторгают клиенты и признают её полезность. Конечно, её можно улучшать и улучшать, но в целом продукт на стадии MVP должен быть полезным, понятным клиенту и таким, чтобы можно было принять решение о его дальнейших улучшениях или признать эксперимент неудачным и тестировать новую гипотезу, потратив при этом как можно меньше ресурсов.

В целом, Lean подход ориентируется на тестирование потребностей и ценностей и попадание в ожидание рынка самыми минимальными средствами. Lean-подход это не о “технических” проблемах и их решениях инженерными средствами, а о предпринимательском подходе к решению задач. Lean предполагает, что ты ищешь самое простое решение для достижения результата: технически, организационно и т.п. упрощая при этом всё что не является действительно важным.

В упрощении сила и главная “ловушка” Lean: стремление всё упростить иногда приводит к ситуациям, в которых продукт упрощают настолько, что теряются действительно важные функции и продукт, по сути, оказывается ненужным, поскольку не несёт ценности пользователю.

В заключение о Lean хочется сказать такое: часто, когда говорят о Lean то говорят, что “Lean это о том, чтобы вместо сервиса сделать лендинг с формой контактов”. Нет, это не Lean. Такое тестирование не противоречит Lean, но Lean-методология предлагает гораздо больше приемов чем этот, и стоит внимательного изучения.

9.5. Основные методы разработки ПО: гибкие методологии

Рассмотрим основные методологии, применяющие гибкий подход к разработке программного обеспечения.

Scrum методология основывается на понятии спринта (sprint), в течении которого выполняется работа над продуктом. Перед началом каждого спринта

проводится планирование (Sprint Planning), на котором производится оценка содержимого списка задач по развитию продукта (Product Backlog) и формирование бэклога на спринт (Sprint Backlog), в рамках которых и действует команда. Для спринта всегда существуют ограничения по времени, обычно от недели до месяца. Жизнь продукта таким образом разбита на равные по продолжительности спринты.

По **Kanban** методологии проект делится на этапы, которые визуализируются в виде канбан-доски. Этапы, это например: планирование, разработка, тестирование, релиз и т.п. Задачи в виде “карточек” перемещаются с этапа на этап. Новые задачи можно добавлять в любое время. Задача закрывается не по истечению конкретного времени, а по смене статуса на “завершено”. Канбан это методология из концепции “бережливого производства”.

RUP (Rational Unified Process) — разработка продукта при данном методе состоит из четырех фаз (начальная стадия, уточнение, построение, внедрение), каждая из которых включает в себя одну или несколько итераций. RUP огромная методология, которую трудно уложить в абзац текста, но методы, рекомендуемые RUP основаны на статистике коммерчески успешных проектов.

DSDM (Dynamic Systems Development Model) — методология, которая демонстрирует набор принципов, predetermined типов ролей и техник.

Принципы направлены на главную цель – сдать готовый проект вовремя и уложиться в бюджет, с возможностью регулировать требования во время разработки. DSDM входит в семейство гибкой методологии разработки программного обеспечения, а также разработок не входящих в сферу информационных технологий.

RAD (Rapid Application Development) — методология быстрой разработки приложений, которая предполагает применение инструментальных средств визуального моделирования (прототипирования) и разработки. RAD предусматривает небольшие команды разработки, сроки до 4 месяцев и активное привлечение заказчика с ранних этапов. Данная методология опирается на требования, но также существует возможность их изменений в период разработки системы. Такой подход позволяет сократить расходы и свести время разработки к минимуму.

XP (Extreme Programming) — методология, которая ориентируется на постоянное изменение требований к продукту и предлагает 12 подходов для достижения эффективных результатов в подобных условиях. Среди них:

- быстрый план и его постоянное изменение;
- простой дизайн архитектуры;
- частое тестирование;

- участие одновременно двух разработчиков в одной задаче или даже за одним рабочим местом;
- постоянная интеграция и частые небольшие релизы.

9.6. Организация процесса разработки

Определившись с методологией управления процессом разработки необходимо также организовать процесс разработки с технической точки зрения. Основой современного подхода к разработке является использование системы контроля версий для управления версионированием ПО, а также для организации взаимодействия между несколькими разработчиками. Одной из самых распространенных систем контроля версий является git, который обладает большим количеством веб-сервисов для хранения совместных репозиторий: GitHub, GitLab, BitBucket и т.д.

Также современная разработка подразумевает использование более сложных, чем обычный Блокнот, инструментов написания кода. Подбор таких инструментов обуславливается предпочтениями конкретного разработчика или же требованиями разрабатываемого проекта.

9.7. Задание

1. Выбрать методологию управления процессом разработки выбранного проекта исходя из потребностей своей команды.
2. Создать удаленный git репозиторий на одном из популярных сервисов (можно использовать другую систему контроля версий при желании).
3. Описать выбранные инструменты разработки программного обеспечения.